RoFormer: Enhanced Transformer with Rotary Position Embedding

Su, Lu, Pan, Murtadha, Wen and Liu (2021; Neurocomputing 2024), arXiv:2104.09864

Paper summary for the CS-781 capstone

The paper at a glance

Citation

J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, Y. Liu. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv 2021 (v5, 2023). Zhuiyi Technology.

One-sentence summary

Encode position by **rotating** each query and key vector by an angle proportional to its position. The attention score then depends only on the *relative* distance between tokens.

Why it matters for our capstone

Our A4 uses RoPE (`Rotary` in `layers.py`). The paper also tests on a **legal** similar-case matching task, close to our Phase 7.

Where it appears in our slides

Main deck: "A4: efficient decoder". Glossary: "Positional information (2)".

Roadmap

1. The problem
2. The basic idea: rotate queries and keys
3. Experiments
4. Assessment and outlook

Attention needs positions; existing methods add them

Self-attention is position-agnostic. Two families of fixes existed:

- **Absolute**: add a position vector to each word vector, $W(x_i + p_i)$. Learned (BERT, GPT) or sinusoidal (Vaswani et al.).
- **Relative**: change the attention score to include the offset $m - n$, e.g. Shaw et al. (2018), Transformer-XL, T5's bias b_{m-n}, DeBERTa.

Relative methods fit language better: "the word two positions back" matters more than "the word at position 417".

Paper's observation: nearly all methods *add* position information to the representation or score. That makes them hard to combine with **linear attention**.

The goal, stated precisely

Find position-aware query and key functions f_q, f_k whose dot product depends only on the words and their *offset*:

$$\langle f_q(x_m, m),\ f_k(x_n, n)\rangle = g(x_m, x_n, m - n)$$

Legal intuition

"Section" followed two tokens later by "302" should look the same whether it appears on page 1 or page 30 of a judgment.

The 2D case: rotation

View a 2D vector as a complex number. Multiply the query at position m by $e^{im\theta}$, and the key at position n by $e^{in\theta}$. That is, rotate them by angles $m\theta$ and $n\theta$:

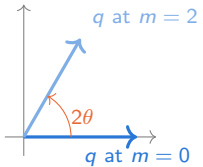
$$f_q(x_m, m) = (W_q x_m)\, e^{im\theta}, \quad f_k(x_n, n) = (W_k x_n)\, e^{in\theta}$$

Their dot product keeps only the *difference* of angles:

$$g = \mathrm{Re}\big[(W_q x_m)(W_k x_n)^* e^{i(m-n)\theta}\big]$$

As a matrix, for position m:

$$\begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix}$$



Rotation never changes a vector's length, only its direction. So RoPE changes *where* a token is, not *what* it says.

Easy worked example: only the offset matters

Let $q = k = [1, 0]$ and $\theta = 30°$ per position.

Case A: query at $m = 3$, key at $n = 5$.

- q rotated by $90°$: [0, 1]
- k rotated by $150°$: $[-0.866, 0.5]$
- $q \cdot k = 0.5$

Case B: shift both by 10, so $m = 13$, $n = 15$.

- q rotated by $390° = 30°$: [0.866, 0.5]
- k rotated by $450° = 90°$: [0, 1]
- $q \cdot k = 0.5$

Result

Both give $0.5 = \cos 60°$, because the offset is 2 positions ($2 \times 30°$) in both cases.

The absolute positions changed, but the attention score did not. RoPE encodes **absolute** position (each vector's own angle) and yields **relative** scores.

Compare with adding positions

With $x + p$, the score contains cross terms such as $x_m^\top W p_n$ that depend on absolute positions. Rotation avoids them.

The general form: many rotating pairs

Split the d-dimensional vector into $d/2$ pairs. Rotate pair i at its own speed θ_i:

$$f_{q,k}(x_m, m) = R^d_{\Theta,m}\, W_{q,k}\, x_m, \qquad \theta_i = 10000^{-2(i-1)/d}$$

$R^d_{\Theta,m}$ is block-diagonal with 2×2 rotation blocks. Then

$$q_m^\top k_n = x_m^\top W_q^\top R^d_{\Theta,n-m} W_k x_n$$

Efficient computation (no matrix needed):

$$R_{\Theta,m} x = x \otimes \cos(m\theta) + \mathrm{rot}(x) \otimes \sin(m\theta)$$

where rot swaps each pair and negates one element: $(x_1, x_2) \to (-x_2, x_1)$.

Speeds for head size $d = 64$

- Pair 1: $\theta_1 = 1$ radian per position. Fast; tells neighbours apart.
- Pair 32: $\theta_{32} \approx 1.3 \times 10^{-4}$. Very slow; tracks long distances.

The same frequency ladder as sinusoidal encodings, but applied by *multiplication*.

Our implementation

`Rotary` pairs dimension j with $j + d/2$ ("rotate half") instead of adjacent dimensions. This is equivalent up to a fixed reordering of dimensions.

Properties of RoPE

1. **Relative by construction**: the score depends on $m - n$ only.
2. **Norm preserving**: rotation matrices are orthogonal, which keeps training stable.
3. **Long-term decay**: with $\theta_i = 10000^{-2i/d}$, an upper bound on the score shrinks as the distance grows. Far-apart tokens interact less, which suits language.
4. **Works with linear attention**: rotation can be applied after the feature maps of linear attention (e.g. Performer), which additive relative methods cannot do.
5. **Flexible length**: no position table, so there is no hard maximum length.

Long-term decay, intuitively

For a nearby token, all 32 pairs are rotated by small, similar angles: their contributions add up. For a distant token, the fast pairs have spun to effectively random angles and cancel out. Only the slow pairs still line up, so the total score tends to shrink.

Caution

"No position table" does not mean it works well beyond the training length. Later work (e.g. the ALiBi paper) found RoPE extrapolates poorly without adjustments.

Translation and pre-training

Machine translation (WMT14 EN–DE, same setup as Transformer-base):

Model	BLEU
Transformer-base	27.3
RoFormer	**27.5**

BERT pre-training (BookCorpus + Wikipedia, 100k steps, batch 64, length 512): replacing BERT's position encoding with RoPE gave **faster convergence** of the masked-LM loss.

Performer + RoPE (Enwik8, 12 layers, length 1024): lower loss and faster convergence than Performer without RoPE.

How strong is this evidence?

- The BLEU gain is small (0.2) and from one run.
- Pre-training results are shown as loss curves, not final numbers.
- All experiments ran on two servers with 4 V100 GPUs each, a modest scale.

The strongest argument for RoPE is its clean mathematics, and its later adoption by many large models.

GLUE fine-tuning: a mixed picture

Model	MRPC	SST-2	QNLI	STS-B	QQP	MNLI (m/mm)
BERT	88.9	**93.5**	**90.5**	85.8	71.2	**84.6/83.4**
RoFormer	**89.5**	90.7	88.0	**87.0**	**86.4**	80.2/79.8

RoFormer is better on 3 of 6 tasks (MRPC, STS-B, QQP) and worse on the other 3.

Honest reading

The authors highlight the three large wins. The table also shows clear losses on SST-2, QNLI and MNLI. This is a good example of why a report should state both sides, as our capstone asks.

Long legal documents: CAIL2019-SCM

Task: 8,964 triplets (A, B, C) of case fact descriptions from the Supreme People's Court of China. Is case B or case C more similar to case A? Most documents exceed 512 characters.

Chinese RoFormer (from WoBERT, word-level tokens) pre-trained on about 34 GB, in stages with maximum lengths from 128 to 1,536.

Model (max length)	Validation	Test
BERT-512	64.13%	67.77%
WoBERT-512	64.07%	68.10%
RoFormer-512	64.13%	68.29%
RoFormer-1024	**66.07%**	**69.79%**

Why this matters to us

This is legal case similarity, very close to our **Phase 7 prior case retrieval**.

- At 512 tokens, RoFormer ties the baselines.
- Allowing 1,024 tokens gains 1.5 test points over RoFormer-512, and 1.7 over WoBERT-512.

Reading *more* of the judgment helps, and RoPE makes the longer input usable.

RoPE in our capstone

- A4 applies RoPE to queries and keys in every layer, with $\theta = 10000$.
- RoPE is applied *before* keys enter the KV-cache, so cached keys keep their positions. Our cache-correctness test checks exactly this.
- With sliding windows, RoPE's relative nature means a key's rotation stays valid however much of the cache is dropped.

Experiments worth running

- A1 with learned positions vs A1 with RoPE, everything else equal.
- Evaluate at 1,024 and at 2,048 tokens to measure extrapolation; compare with ALiBi (A3).
- Phase 7: does a longer context help case retrieval, as on CAIL2019-SCM?

Strengths and limitations

Strengths

- Elegant derivation: relative scores from absolute rotations.
- No learned position parameters.
- Norm-preserving and cheap to compute.
- Compatible with linear attention and KV-caching.
- Became the standard in many modern large language models.

Limitations

- The authors admit they cannot fully explain why it converges faster, or why it does better on long texts.
- Small-scale experiments, few repeated runs.
- Mixed GLUE results.
- Extrapolation beyond the trained length is weak without extra techniques.
- Needs an even head dimension, and the base 10000 is a hand-set choice.

Future research directions

What followed

- Adopted by GPT-NeoX, PaLM, LLaMA and many others.
- **Context extension**: Position Interpolation (Chen et al., 2023) squeezes long inputs into the trained angle range; YaRN (Peng et al., 2023) rescales frequencies unevenly.
- **Better extrapolation**: xPos (Sun et al., 2023) adds an exponential decay to RoPE.
- **Comparisons**: ALiBi (Press et al., 2022) reported better length extrapolation than rotary embeddings.

Open questions

- A theory of *why* RoPE trains faster (flagged by the authors).
- The best frequency base θ for a given context length.
- How position schemes affect retrieval of facts from far back in a document.
- For legal NLP: do judgments need longer context than the 1,024 tokens used here? How far back do case references reach?

Summary

- Earlier position methods *add* position to representations or scores.
- **RoPE** *rotates* each query and key pair by angle $m\theta_i$. The dot product then depends only on the offset $m - n$.
- Worked example: positions (3, 5) and (13, 15) give the same score, 0.5.
- Properties: relative, norm-preserving, long-term decay, linear-attention compatible, no position table.
- Results: small gains in translation, faster pre-training convergence, mixed GLUE, and better accuracy on long legal case matching with 1,024 tokens (69.79% test).
- For us: the position scheme of A4.

Reference

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864 (v5, November 2023). Published in *Neurocomputing*, 2024.

Other work cited: Shaw et al. (2018); Xiao et al. (2019) CAIL2019-SCM; Choromanski et al. (2020) Performer; Press et al. (2022) ALiBi; Chen et al. (2023) Position Interpolation; Peng et al. (2023) YaRN; Sun et al. (2023) A Length-Extrapolatable Transformer (xPos).